JARBY JESSID COSME

Software & AI Engineer | Production AI Systems, Backend & Data

Cali, Colombia · jessidc95@gmail.com · English B2 (upper-intermediate)
linkedin.com/in/jarbyjessidcosme · github.com/JessidC · Portfolio: jarby.dev
Available for full-time remote US nearshore roles (GMT-5, overlaps with EST/CST)

PROFESSIONAL SUMMARY

Software engineer with ~7 years of experience (Python, C#/.NET, TypeScript, SQL) who takes systems from zero to production. I'm the sole technical owner of SIESA's AI support platform, built to improve the whole service. I started from the data, built and run the RAG that answers the ERP support chat in every module, and measure what actually gets resolved with LangGraph agents running in production. I present business cases to the CEO. My public repos rebuild the evaluation method and the learning loop on open data, with numbers anyone can rerun.

PROFESSIONAL EXPERIENCE

Software Developer — SIESA, Cali, Colombia
Mar 2022 – Present

AI support platform — sole technical owner (2025 – Present)

- **Diagnosis first:** analyzed 90 days of production conversations and presented the business case for an in-house engine to leadership, with 6 proposals ranked by impact and cost.
- **A RAG built for the service:** designed, built and operate the RAG platform behind the support chat, on three rules: never make things up (citation and numeric validation, deterministic guards), hand off to a person on low confidence (tool calling) and never brush the customer off. Human escalation per chat in Payroll dropped, confirmed by a significance test, and replies that send customers to "contact support" became rare. *Framework-free Python on the critical path, a large and a small managed LLM, vector search with reranking*.
- **Quality measured, not assumed:** golden sets with an LLM judge gating releases, A/B replay on real traces and significance tests. Answer correctness rose between evaluation cuts (human-validated judge) and prompt caching lowered the cost per turn. Measured hybrid BM25 + RRF retrieval twice and retired it: the lever was the content (reprocessed the document base with multimodal OCR).
- **Measuring what gets resolved — Jarvis:** built Jarvis, a daily LLM judge that decides who resolved each conversation (AI, agent or nobody) with the same rule for both. It grew out of the judge I already ran, replaced the previous ticket classifier and re-types every ticket at close, in production.
- **LangGraph in production:** Jarvis's closing engine and the service's nightly processes (resolution judge, scoring, ticket typing and expert-approved curation) run as LangGraph and LangChain graphs I built, with typed state, parallel judges, PostgreSQL checkpoints to resume a run and human-review interrupts. Also rewrote the bot pipeline as a graph, with parity proven on its full test suite.
- **Built for others to build on:** a self-improvement loop with one-click expert approval that adds new Q&As every week and helped lower Commercial escalation, an admin console with per-module permissions and a model gateway other teams use. Run production with per-stage observability (p50/p95 latency, cost per turn) and module-by-module releases with rollback plans.
- **The service, not just the model:** redesigned support operations (per-agent WIP limits, FIFO assignment, load balancing). Customer satisfaction rose clearly in the weekly survey, and first response time and peak load per agent went down.
- **What's next:** designing an online triage channel that sends each request to AI, L1, L2 or L3, and building RAG v2, which will act on customer systems with per-action permissions. Trained 16 people across 3 teams on AI adoption.

Internal AI assistant & automation (2024 – 2025)

- Built the internal AI assistant, an LLM with tool calling over internal databases, the Jira-style platform and support tickets, so support agents, consultants and operations asked in plain language instead of opening several systems. Prototyped in LangChain, then built straight on the API.
- Automated the operational loop with n8n (alerts to agents on tickets close to their SLA, TTR and FCR targets) and Power BI dashboards fed from the Jira-style platform.

Data, automation and platforms (Mar 2022 – Present)

- Replaced manual reporting with a Python pipeline (ticketing API → SQL Server with hash integrity checks → Power BI via REST API), validated against the manual load before replacing it, and cut a daily SQL Server load time by more than half with identical results.
- Lead the in-house service and project management platform (Jira-style) used across the company; integrated 3 applications. Migrated 3 monoliths to microservices (.NET 8 / ASP.NET Core) in an agile team and modernized React/Angular modules.

Independent AI Consultant & Instructor — Part-time, remote 2026 – Present

- Founded and build Tocaturno (tocaturno.com), a production SaaS for 24/7 WhatsApp reception (bookings, reminders and handoff to a person for sensitive cases). Design AI agents and automations for clients (WhatsApp, n8n, MCP servers) and teach hands-on AI classes.

Freelance Software Developer — Self-employed, remote Jun 2019 – Feb 2022

- Built 6+ full-stack web applications for Colombian SMBs (.NET Core, React, JavaScript; 1,500+ end users), including a PayU e-commerce platform (US$25K+ in first-year sales) and a multi-site inventory system (stock discrepancies –90%).

PUBLIC PROJECTS

- **rag-eval-bench** (github.com/JessidC/rag-eval-bench). A simpler public version of my production RAG's release check, on the Odoo 18 docs: a golden set with unanswerable questions, an LLM judge, a citation and number validator, and a McNemar test as a CI gate. Hybrid retrieval beat dense by 12.5 points ($p = 0.02$; 9.7, not significant, with an out-of-family judge), and a reranker that only added latency didn't ship.
- **support-learning-loop** (github.com/JessidC/support-learning-loop). The loop where agents teach the bot, rebuilt on the ABCD dataset, with an out-of-family judge and checks that reject anything the agent didn't say or only a person can do. On 330 held-out chats, correct resolutions went from 19% to 59% and handoffs from 68% to 24% ($p < 0.0001$).
- **enterprise-fullstack-dotnet** (github.com/JessidC/enterprise-fullstack-dotnet). .NET 8 reference monorepo: layered ASP.NET Core API, JWT with roles, PostgreSQL, React + Vite, tests, Docker and GitHub Actions CI.

TECHNICAL SKILLS

- **AI & LLM:** RAG • AI Agents • LangGraph / LangChain • Tool Calling • MCP • Embeddings & Semantic Search (pgvector) • Reranking • LLM Evaluation (LLM-as-Judge, golden sets, A/B) • Guardrails & Answer Validation • Claude (AWS Bedrock) • Azure OpenAI / AI Foundry • OpenAI API • n8n • Claude Code, Cursor
- **Backend & Frontend:** Python • FastAPI • C# • .NET 8 • ASP.NET Core • REST APIs • WebSockets • Microservices • Clean Architecture / SOLID • Testing (pytest, xUnit/NUnit) • React • TypeScript • Angular
- **Data & Cloud:** SQL Server (T-SQL) • PostgreSQL / pgvector • ETL Pipelines • Medallion Architecture • Power BI • AWS (Bedrock, EC2, S3, IAM) • GCP (Cloud Run) • Linux • Docker • CI/CD • Git

EDUCATION

- **B.Sc. in Systems Engineering**, UNAD, in progress (expected 2027) • **Technologist in Systems Analysis and Development**, SENA, 2022